3. AI Large Model Voice Interaction

1. Concept Introduction

1.1 What is "AI Large Model Voice Interaction"?

In the `largemodel` project, **AI Large Model Voice Interaction** connects the previously introduced **Offline ASR** and **Offline TTS** with the **Large Language Model (LLM)** core to form a complete conversational system that can listen, speak, and think.

This is no longer an isolated function, but the prototype of a true **voice assistant**. Users can have natural language conversations with the robot through voice, and the robot can understand questions, think of answers, and respond with voice. The entire process is completed locally without the need for a network.

The core of this function is the `model_service` **ROS2 node**. It acts as the brain and nerve center, subscribing to the recognition results of ASR, calling the LLM to think, and then publishing the LLM's text reply to the TTS node for speech synthesis.

1.2 Brief Description of the Principle

The implementation principle of this function is a classic data flow pipeline:

1. **Sound -> Text (ASR)**: The `asr` node continuously listens to the ambient sound. Once it detects that the user has finished speaking a sentence, it converts it into text and publishes it to the `/asr_text` topic.
2. **Text -> Thinking -> Text (LLM)**: The `model_service` node subscribes to the `/asr_text` topic. When it receives the text from ASR, it sends it as a prompt to the locally deployed large language model (such as `Qwen` run through `Ollama`). The LLM will generate a text response based on the context.
3. **Text -> Sound (TTS)**: After the `model_service` node gets the LLM's response, it publishes it to the `/tts_text` topic.
4. **Sound Playback**: The `tts_only` node subscribes to the `/tts_text` topic. After receiving the text, it immediately calls the offline TTS model to synthesize it into audio and play it through the speaker.

This process forms a complete closed loop of **voice input -> text processing -> text output -> voice output**.

2. Project Architecture

2.1 Key Code Analysis

The core of the entire process is how the `model_service` node connects the input and output.

1. Subscribe to ASR results (in `largemodel/model_service.py`)

The `model_service` node will have a subscriber to receive the text recognized by ASR.

```
# largemodel/model_service.py (core logic demonstration)
class ModelService(Node):
    def __init__(self):
        super().__init__('model_service')
        # ...
        # Subscribe to the text output topic of ASR
        self.asr_subscription = self.create_subscription(
            String,
            'asr_text',
            self.asr_callback,
            10)

        # Create a publisher for the text input topic of TTS
        self.tts_publisher = self.create_publisher(String, 'tts_text', 10)

        # Initialize the large model interface
        self.large_model_interface = LargeModelInterface(self)
```

Explanation: The `__init__` method of the node clearly defines its role: a middleman that can "listen" to ASR results, "command" TTS to speak, and has an internal "brain" (`LargeModelInterface`).

2. Process ASR text and call LLM (in `largemodel/model_service.py`)

When ASR has a new recognition result, `asr_callback` will be triggered.

```
# largemodel/model_service.py (core logic demonstration)
    def asr_callback(self, msg):
        user_text = msg.data
        self.get_logger().info(f'Received from ASR: "{user_text}"')

        # Call the large model interface to think
        # llm_platform determines whether to call Ollama or an online API
        llm_platform = self.get_parameter('llm_platform').value
        response_text = self.large_model_interface.call_llm(user_text,
llm_platform)

        if response_text:
            self.get_logger().info(f'LLM reply: "{response_text}"')
            # Send the LLM's reply to TTS
            self.speak(response_text)
```

Explanation: This is the core logic of the system. After the callback function receives the text, it immediately sends it to the LLM through `large_model_interface`. The `call_llm` method will decide whether to connect to the local Ollama or an online API based on the configuration of `llm_platform`.

3. Send LLM reply to TTS (in `largemodel/model_service.py`)
The `speak` method is a simple encapsulation used to publish text to the topic that the TTS node is listening to.

```
# largemodel/model_service.py (core logic demonstration)
    def speak(self, text):
        msg = String()
        msg.data = text
        self.tts_publisher.publish(msg)
```

Explanation: This function completes the final step of the data flow, passing the text result thought out by the "brain" to the "mouth", thus completing the entire voice interaction loop.

3. Practical Operation

3.1 Configure Online LLM

1. **Update the key in the configuration file and open the model interface configuration file `large_model_interface.yaml`**:

   ```
   vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
   ```

2. **Fill in your API Key**:
   Find the corresponding section and paste the API Key you just copied. Here is an example of Tongyi Qianwen configuration.

   ```
   # large_model_interface.yaml

   ## Tongyi Qianwen
   qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
   qianwen_model: "qwen-vl-max-latest" # You can choose the model as needed,
   such as qwen-turbo, qwen-plus
   ```

3. **Open the main configuration file `yahboom.yaml`**:

   ```
   vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
   ```

4. **Select the online platform to use**:
   Modify the `llm_platform` parameter to the name of the platform you want to use.

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi'  # Optional platforms: 'ollama', 'tongyi',
'spark', 'qianfan', 'openrouter'
```

After modifying the configuration file, you need to recompile and source in the workspace:

```
cd ~/yahboom_ws
colcon build && source install/setup.bash
```

3.2 Start and Test the Function

1. **Open a new terminal and start the `largemodel` main program**:
   Run the following command to start voice interaction:

   ```
   ros2 launch largemodel largemodel_control.launch.py
   ```

2. **Test**:
   - **Wake up**: Say to the microphone: "Hi,Yahboom."
   - **Conversation**: After the speaker responds, you can say what you want to ask.
   - **Observe the log**: In the terminal where the `launch` file is running, you should see:
     1. The ASR node recognizes your question and prints it out.
     2. The `model_service` node receives the text, calls the LLM, and prints the LLM's reply.
   - **Listen to the answer**: After a while, you should be able to hear the answer from the speaker.